

Crane & Cargo

Project documentation

Technical presentation and user manual

Interactive Graphics • Sapienza University of Rome

Team: Gandolfi Alessia, Iacomì Gabriel, Robotti Marcello

Contents

1	What the project is	2
2	Environment used	2
3	Libraries, tools and models not developed by the team	3
3.1	Libraries and tools	3
3.2	3D models (external)	3
3.3	Textures (external)	4
4	User manual	4
4.1	First start and screen	4
4.2	Camera views	4
4.3	Crane, cargo, ship and truck	4
4.4	Day, night and walking	5
5	Technical aspects	5
5.1	Animations: tween, delta time and the crane	5
5.2	Physics and collisions	6
5.3	Lighting: day, night and sky	7
5.4	Atmosphere: fog and sea	8
5.5	Dock platform and road network	9
5.6	Container areas	9
5.7	Walkable worker	9
5.8	Camera system	10
5.9	Sound	10
5.10	Static decorations	10
6	Implemented interactions	11
7	Design choices	11
7.1	Limitations and simplifications	11

7.2 Conclusion 12

This text describes **Crane & Cargo** in two ways. First as a technical report: what we used and how the harbor is put together. Second as a user manual: how to open the application and which keys do what. Full asset URLs and authors are in the repository `README.md`.

Play the project

Repository:

<https://github.com/sapienzainteractivegraphicscourse/final-project-denata>

Live demo:

<https://sapienzainteractivegraphicscourse.github.io/final-project-denata/>

Local run (Node.js): `npm install` then `npm run dev`. Open the Vite address, usually `http://localhost:5173`. Wait until loading ends. Production build: `npm run build`. Dependencies are locked in `package-lock.json`.

1 What the project is

We built a 3D harbor that runs in the browser. The player works as a crane operator. A cargo ship docks. You pick containers with a portal crane, stack them on the dock, put them back on the ship, or load them on a truck that leaves the port.

Around this loop we added a full port: roads, buildings, decorative ships, workers, day and night lighting, fog, and physics when a container falls. There is no score and no timer. It is a sandbox.

All animations are written in JavaScript. We did not import animation clips from the models. The crane mesh itself was not usable as-is: we reworked it in Blender so the code could move boom, cables and spreader as separate parts (Section 5.1).

The main engineering problem is that the same objects live in several representations at once. A container can be a visual mesh, a slot occupant, a child of the spreader, or a Cannon rigid body. Only one of those layers owns the transform at a time.

2 Environment used

The project runs in a web browser. The graphics API is **WebGL**.

We did not write the whole renderer in raw WebGL. We used **Three.js** (0.185). It gives a scene graph, cameras, lights, materials, loaders, bounding boxes and vector/matrix math. The renderer is `WebGLRenderer` with antialiasing. Shadows use `PCFSoftShadowMap`.

```
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;
```

The page is plain HTML and CSS. The code is JavaScript modules. **Vite** (8) is the build tool. The entry point is `index.html`, which loads `src/main.js`. Deployment is GitHub Pages.

Each main object (ship, crane, truck, dock, worker) is a class with a `THREE.Group` called `root`. `main.js` creates the scene, reads the keyboard, and runs the game loop. The camera is a `PerspectiveCamera` (FOV 70, near 0.1, far 1000) with `OrbitControls` and a bit of damping.

3 Libraries, tools and models not developed by the team

3.1 Libraries and tools

Name	Version	What we use it for
three	0.185.1	Scene, renderer, lights, materials, GLB loading
@tweenjs/tween.js	25.0.0	Timed motion for the ship, the truck and decorative ships
cannon-es	0.20.0	Physics for falling containers
vite	8.2.1	Dev server and production build

Three.js add-ons: `GLTFLoader`, `OrbitControls`, `Water`, `BufferGeometryUtils`. We also used Node.js / npm, Git, and GitHub Pages. **Blender** is not a runtime library, but it was a real part of the pipeline: we used it to split and rename the harbor crane before export.

3.2 3D models (external)

We found these models online. We scaled them, placed them, and animated them in code. We did not model them from scratch, and we did not import their animations. The crane is the exception: the downloaded file was edited in Blender (Section 5.1). Full source URLs are in the repository `README.md`.

Gameplay objects

File	What we use it for
Crane_Modified3.glb	Portal crane (named parts after Blender)
trucks.glb	Truck with wheels FR, FL, RL, RR
Worker.glb	Walkable worker (skinned mesh)
flashlight.glb	Flashlight the worker can hold at night
ships/ship_1.glb	Playable cargo ship, first type
ships/ship_3.glb	Playable cargo ship, second type
ships/deco/deco_ship_1..4.glb	Background ships

Containers. Several GLB files are picked at random, then every clone is scaled to the same size: `20ft_container.glb`, `cargo_container_new.glb`, `container.glb`, `container_3d_model.glb`, `sea_cargo_container_-_legendarygamedev.glb`, `shipping_container.glb`, `shipping_containers.glb`.

Environment and props

File	What we use it for
industrial_buildings_set_-_low_poly_models.glb	Warehouses, tanks, power plant, silos
street_lamp.glb	Street lamps along the roads
forklift_low_poly.glb	Two parked forklifts
fuel_truck.glb	Decorative fuel truck
docks_bollard.glb	Mooring posts on the quay
workers.glb	Static worker poses around the yard
damaged_chainlink_fence.glb	Fence sections
oil_barrel_opt.glb	Oil barrel piles
plastic_water_container_-_4mb.glb	Plastic water containers

3.3 Textures (external)

Dock: Poly Haven *Brushed Concrete 2* (diffuse, normal, roughness). Sea: Three.js `waternormals.jpg`. Roads: asphalt tile plus paint and wear decals (dashes, edges, STOP, arrows, yard letters, cracks, oil, dirt, dust, salt, net, fish). Roads, the dock box, the sky dome and the fog walls are built in code.

4 User manual

4.1 First start and screen

Open the GitHub Pages link or run locally. A dark loading screen stays until models and shaders are ready. Keyboard and camera buttons are off until then.

Default camera: overview. Left mouse orbits, wheel zooms, right mouse pans if the view allows it.

- **Top left:** main keys. **Top center:** prompts such as “Press E to load the container”.
- **Top right:** Day / Night, and Sunrise (day only). **Bottom left:** camera views 1–7.

If a container blocks the truck road: *Move the container blocking the road so the truck can pass.*

4.2 Camera views

There is only one camera. Changing view only changes pose and limits.

Key	View	What you see
1	Overview	Working area. Mouse orbit on.
2	Crane	Follows the crane on the rails.
3	Cabin	First person toward the spreader. Orbit off.
4	Ship	From the sea toward the quay.
5	Truck	High view of the road and depot.
6	Worker	Third person behind the walkable worker.
7	Worker FP	First person from the worker.

The blend takes about 0.2 seconds. In views 6 and 7 you control the worker. Crane keys do nothing until you leave those views.

4.3 Crane, cargo, ship and truck

W / **S** boom, **A** / **D** rotate, **arrows** travel along the dock, **R** / **F** hoist the spreader. Hold to keep moving. If the spreader or the held box hits cargo or the truck, that axis stops. Other axes still work.

E pick (spreader above the box) or place on the nearest free ship/depot slot. **C** load the parked truck. **Q** drop; physics takes over.

A typical flow: SPACE for the ship, lower the spreader, E, then E / C / Q, then T if you need the truck.

SPACE docks or sends the cargo ship (only when fully docked or fully gone). If a decorative ship is crossing, the game waits. **T** calls or sends the truck. After C, or after you pick the truck cargo, the truck leaves by itself. On return there is a 20% chance of a new box. If the road is blocked, the truck waits.

4.4 Day, night and walking

Day: bright sky and sun shadows. Night: lamps, crane lights, truck lights, red obstruction beacons on tall structures, no sun shadows. Sunrise (day only): warmer light and a low sun. Dock ambient sound starts after the first click or key. A warning beep plays while the crane travels along the dock. Views 6–7: arrows walk and turn inside a limited yard. **L** flashlight at night. Keys 1–5 return to crane / overview cameras.

5 Technical aspects

The code lives in `src/`. Each file looks after one topic: the dock, the crane, the ship, lighting, and so on. That made it easier to work as a team.

Every frame the program reads the keys, moves either the crane or the worker, updates the ship and the truck, checks cargo prompts, steps physics, moves the water a little, updates the camera, and draws the picture. Time comes from `requestAnimationFrame`. We turn it into seconds and clamp it, so a slow frame does not make the crane jump.

The same arrow keys do two jobs. In a worker view they walk. In a crane view they move the crane along the dock. They never do both at once.

At start we wait until water, dock, truck, crane, ships, worker and cargo are loaded. Then we warm up the shaders for day, night and the flashlight, so the first switch to night does not freeze. The loop starts only when everything is ready.

5.1 Animations: tween, delta time and the crane

We use two kinds of motion on purpose.

Tween.js is for things that already know where they are going and how long they should take. The cargo ship moves in, waits, turns, and leaves. There are two ship models and two routes. The truck follows a path made of straight pieces and rounded corners that match the road. Arrival and departure takes about sixteen seconds. Decorative ships cross the sea in about 23 seconds. A new one can appear every 15 seconds if the main ship is not moving.

We update those tweens ourselves in the game loop. That way a truck that is waiting for a blocked road does not keep counting time while it is stuck.

Delta time is for things the player holds. The crane boom, the rotation, the travel along the dock, the hoist, and the worker walk all use it. When you release a key, that motion must stop at once. The same is true if a collision test fails. A tween would keep going for a set duration. That would feel wrong on the crane.

The crane The crane is the main hierarchical model. The file we downloaded looked good, but many parts were joined. If we had rotated the boom, bits that should stay still would have moved with it. So we edited the model in Blender before we put it in the game.

We split the mesh into boom, platform, piston, six upper cables, the vertical cable bundle, spreader, tower and wheels. We renamed those parts so the code can find them after loading. We also raised the outriggers. In the original file they were down, as if the crane were parked for a long job. We needed a crane that can roll along the quay. Then we exported `Crane_Modified3.glb`.

In the running scene the platform is the parent of the moving upper body. The boom rotates from

the keys. The piston and the upper cables must stay connected to both the platform and the boom, so they cannot be simple children. Each frame we look at the two ends, point a small controller from one to the other, and scale it along its length. The cable mesh looks longer or shorter, but it is the same geometry.

The vertical cables are a special case. If they were glued to the boom they would tilt with it, which looks wrong. The top follows the boom. The bottom stays under that point and keeps its height, so the bundle hangs straight. The spreader follows that lower point and does not copy the boom tilt. That is the hook you hang a container on.

```
createStretchablePart(mesh, start, end, startAnchor, endAnchor) {  
  const controller = new THREE.Group();  
  this.parts.Platform_Body.add(controller);  
  this.pointController(controller, start, end, 1);  
  controller.attach(mesh);  
  return { controller, startAnchor, endAnchor,  
    originalLength: start.distanceTo(end) };  
}
```

We did not import a crane animation. Speeds are written in the code. The boom moves at about 12 degrees per second, and we clamp it between -70° and -5° so it cannot fold into the tower or point straight up. The upper body turns at 15 degrees per second. Travel along the dock is 1.5 units per second. The hoist is 3 units per second. If the held box is already on the ground, F does not push it through the dock.

Each crane movement is tried, then checked. We measure overlap, apply the move, update the cables, and measure again. If the overlap got worse, we undo that axis only. You can still rotate while the hoist is blocked. The twelve wheels sit on small pivots and spin from the distance travelled, so they look like real wheels and not meshes turning around the wrong point.

When we hang a container on the spreader we reparent it with **attach**, so it does not jump in the world. We also undo the crane model scale. Otherwise the box would shrink with the GLB.

5.2 Physics and collisions

Three.js draws what you see. Cannon moves invisible boxes. We did not turn every triangle of every building into a collider. That would be slow and unstable. Only the things that matter for play have a simple shape.

A container that sits in a slot, or hangs from the crane, is kinematic: the mesh tells the physics body where to be. A container you drop with Q is dynamic: Cannon applies gravity and contacts, then we copy the pose back to the mesh. The dock and the ship hull are static. Distant props, the water and the decorative ships have no physics at all.

Gravity is normal Earth gravity (9.81). There is no bounce: restitution is zero. Friction is fairly high (0.6), and dropped boxes have a bit of damping so they settle instead of sliding forever. We also cap how fast a box can fall, so a long drop does not explode through the dock. The physics step still runs at 60 Hz. If a frame is slow, Cannon can take extra substeps (up to ten) to catch up. Cargo can hit the ground, the hull, the crane and the truck. The dock does not need to collide with the crane body. That saves work.

The visual mesh of a container sits on its bottom. The physics box is centred, so we raise the body by half the height. If we forget that, the box floats or sinks.

```
if (state === 'free') {
```

```
body.type = CANNON.Body.DYNAMIC;
body.mass = 500;
} else {
body.type = CANNON.Body.KINEMATIC;
body.mass = 0;
}
```

Cannon is good at falling boxes. It is not good at stopping two kinematic objects. The crane is kinematic and a stack is kinematic, so the engine would let the spreader pass through a container. We added our own box tests in Three.js. We test the spreader and the held cargo against other containers and the truck. We do not test the boom: that box would cover half the port. We do not test the whole ship hull either: it covers the hold and would freeze the crane. The hull is only there so a dropped box can land on the deck.

The truck is two boxes, not one. A single tall box as high as the cabin would swallow cargo sitting on the bed. We use a short tall cab and a long low bed.

Before the truck drives, we walk along its path every few metres and place those two boxes. If a container is in the way (except the one on the truck itself), the truck waits and a message appears. The tween clock is paused, so the truck does not skip ahead when the road is free again.

A falling box can knock a stack on the dock. We listen to Cannon contacts, and we also check boxes after the step, because contacts are sometimes missed. If the stack is allowed to fall, we always free the top container. Cargo on the ship cannot be knocked off. That was a choice: a messy hold looked worse than a slightly less realistic one.

What physics does not do

A held container is not hanging from a physical rope. It follows the spreader until you press Q. Keys are not locked; we undo a bad move after we try it. Cables do not go slack. The worker has no physics body. Decorative ships do not bump the playable ship. We just wait a few seconds on SPACE.

5.3 Lighting: day, night and sky

Day, sunrise and night are not three different scenes. It is the same harbor. One function, `applyLook()`, changes the lights, the sky colours, the fog, the water and the lamps.

By day a hemisphere light fills the shadows a little, and a directional sun does the real lighting and the shadows. The hemisphere is weaker than the sun on purpose. If ambient light is too strong, you lose the shadows. The sun shadow map is large and aimed at the working area. We do not move that volume with the camera. When we tried that, shadows flickered while you orbited.

Sunrise is only available in day mode. The sun sits lower and the colours go warmer. Night turns the sun off. A pale moon light from the other side keeps the port readable, but it does not cast shadows. A second shadow map at night was too expensive for what it gave us.

Street lamps were a problem. One real point light per pole, with shadows, killed the frame rate, and the cones looked fake. Now the glass becomes emissive at night, we draw small bright bulbs, and we put a cheap glow on the ground under each lamp. You still read them as lamps. The scene is not really lit by dozens of lights.

The crane has its own lights, parented to moving parts, so they travel with the cabin and the spreader. The truck has headlights and taillights parented to the truck, so they turn with the road.

At night we also switch on red obstruction beacons, the kind you see on masts and chimneys so aircraft can spot them. They sit on the tall dock buildings, on the crane tower and boom tip, and on the masts of the playable and decorative ships. They are not real lights. They are small glowing points that share one material. Fog is off on that material, so they stay red in the distance. Every frame we pulse their size a little, so they look like they blink. By day the same points are simply hidden.

Sky We did not use a photo skybox. The sky is a large sphere around the port. We look at the inside of it. Fog is turned off on that material. If fog faded the sky, the horizon would go grey and the gradient would die.

The shader looks at how high you are aiming. Straight up is one colour, the horizon is another, and we mix them softly so there is no hard band. At sunrise we add a glow only on the sun side, low on the horizon. At night the same idea uses the moon direction, so one side of the sky still feels a bit brighter. We never rebuild the sphere. We only write new colours into it.

The background colour of the scene matches the fog, so any gap at the edge of the dome does not show a different tint.

5.4 Atmosphere: fog and sea

The dock and the sea are huge. Without fog you see the rectangular end of the world. We ended up with two fog ideas that work together.

The first is normal distance fog. Far objects fade into a colour that follows the time of day: cooler at noon, greyer at sunrise, darker at night. That helps, but on the sides you could still see a sharp dock edge.

The second is a ring of transparent walls around the working area. There are several layers, more transparent on the inside, denser on the outside. They are a bit irregular, so they do not look like a perfect cylinder. Toward the open sea the ring sits farther out, so it does not hide the ships. Toward the berth we keep a clearer channel. The walls fade at the ground and at the top, so they look more like mist than a fence. They do not use the same distance fog. If they did, they would vanish in the distance and the trick would fail.

When you switch day, sunrise or night, we only change the colour of those walls. We do not rebuild them.

Why two fogs

Distance fog fades far buildings, water and ships. The wall ring hides the geometric edge of the dock. One system cannot do both well.

The sea is the Three.js water helper with a moving normal map. We advance its time every frame so the waves move. It follows day and night, but we were careful with the specular colour. If we used the same yellow as the scene sun, the water looked dirty. By day the highlights stay bluish. We also lowered the reflection, because the default look was too shiny. The sea receives sun shadows but does not cast them. Dropped cargo does not float. Water is only a picture.

5.5 Dock platform and road network

The land is a very large box. We made it this big so that, from the crane, you do not see a hard edge. The top is concrete with a colour map, a normal map and a roughness map. Concrete is not metal, so metalness is zero. A small shader trick rotates each texture tile, otherwise the dock looks like repeating wallpaper. We also desaturate the colour a bit and tint it toward the sea.

The roads are not an imported mesh. They are boxes and quarter-circles we built in code, so they match the truck path. Asphalt is a repeating tile. On top we draw white dashes, edges, stop bars, arrows, yard letters, and wear: cracks, oil, dirt, dust. Those marks are what make the ground look used.

Large ground pieces receive shadows but do not cast them. If the huge dock cast a shadow, it would darken the whole scene.

5.6 Container areas

All interactive containers share one size, about as large as a 20-foot box. We load several different models and clone a random one. If one file fails to load, the others still work. Then we scale the clone so stacking stays stable.

There are three places where the crane can really put a box:

- the ship, 12 or 24 slots depending on the ship type, stacks of two. Those stacks cannot be knocked off.
- the depot next to the crane, a grid of 12 slots, stacks of three.
- the truck, one slot, one box.

Around the yard there are extra stacks. Those are only decoration. The crane does not place into them.

A slot is just an empty object in the scene, an anchor. When we place a container, it becomes a child of that anchor. Height depends on how many boxes are already there. The crane does not need to know if the slot is on a ship or on the dock. It only talks to cargo areas.

Picking is stricter than a simple distance. The spreader must sit above the box, not beside it, and there must be room to hang the container. If the lifted box would hit a neighbour, we hide the E prompt. That avoids snapping one container into another pile.

Only one system moves a box at a time. In a slot, the slot owns it. On the crane, the spreader owns it. After Q, Cannon owns it. When you place it again, the new slot takes over.

5.7 Walkable worker

The worker is a skinned character. We scale it to about human height and put the feet on the ground. We did not import a walk clip. While you hold the arrows, we swing the legs and the opposite arms in code. In first person we raise the arms a little so you see less of a T-pose.

You can only walk in a limited part of the yard. There is no physics body and no collision with containers. It is a way to look at the port from eye height, not a second player that can knock stacks.

The flashlight is a small model plus a spot light. We copy the right-hand pose each frame so it sits in the hand. It only works at night, in a worker view. Leaving night puts it away.

5.8 Camera system

There is only one camera. The seven views are seven ways to place it.

Overview, ship and truck are more or less fixed. Crane view follows the crane on the rails, not the rotating boom. If the camera were a child of the upper body, turning the crane would spin the view with the arm, and that was hard to use. Cabin view sits in the tower and looks toward the spreader. We push it forward a few metres, otherwise you sit inside the mesh and see only a wall. Worker third person sits behind the character. Worker first person is from the eyes.

Switching view takes a short blend, about a fifth of a second, so the jump is not harsh. Follow views are updated after the crane moves, so you land on this frame's pose, not an old one. Pan is limited so you cannot drag the target into the sky or under the dock. Cabin and worker first person lock the mouse orbit. The pose comes from code.

5.9 Sound

There are two loops, both ordinary HTML audio files. One is dock ambience. Browsers keep audio muted until the page gets a real click or key, so we try to start it at load and again on the first pointer or keyboard event. After that it stays in the background.

The other is a crane warning beep. It plays only while the crane travels along the dock (the up and down arrows). When you stop, the volume fades out and the beep pauses. Boom, rotation and hoist stay silent. That keeps the sound tied to the moving machine, not to every key.

5.10 Static decorations

Most of the port is not interactive. Warehouses, tanks, silos, parked forklifts, a fuel truck, bollards, fences, oil barrels and water tanks are there so the place feels like a working harbor.

We instance fences, barrels and some props: many copies, few draw calls. Street lamps are placed along the roads. At night they glow as described above, and the red beacons on the tall buildings come on with them. Decorative workers are static poses: someone with a toolbox, someone seated on a forklift, and so on. They do not walk.

Background ships are the same idea on the water. Some sit docked. Others cross in the distance on a tween. They make the sea look busy, and they are why we wait on SPACE: we do not want two hulls to occupy the same berth.

None of this is simulated. If a container falls into a fence, nothing special happens. The decorations are the set. The play is the crane, the slots, the truck and the dropped boxes.

6 Implemented interactions

Interaction	How
Orbit, zoom, pan	Mouse, depending on view
Viewpoint / look	Keys 1–7 or buttons; Day / Night / Sunrise UI
Ship / truck	SPACE / T (truck also after load)
Crane / cargo	W S A D, arrows, R F; E pick/place; C truck; Q drop
Worker / light	arrows in 6–7; L flashlight at night
Sound	dock loop after first click; beep while the crane travels
Blocked road / knocks	automatic wait; Q and collisions

7 Design choices

We used Three.js because after some analysis, we thought that it would be easier than raw WebGL for a scene this size.

Tween.js is used for the ship and the truck because those paths last many seconds. The crane uses delta time because the player holds keys and must stop at once on a collision.

We added Cannon because dropped containers should fall and bounce. We still needed our own box tests for the crane and the truck. Cannon does not stop two kinematic objects, so a held box would go through a stack if we trusted the physics engine alone.

Roads are procedural so the truck path matches the mesh. Containers come from many models but one size, so stacking stays stable.

Ship cargo is not knockable. A falling box on the quay can topple a depot stack; a box on the ship should stay in the hold. Night lamps do not cast shadows. It was too slow, and the cones looked fake. We light the glass and put a cheap glow on the ground instead.

We edited the crane in Blender because the downloaded file was one visual mesh, not a machine we could animate. After the split, JavaScript can rotate the boom and stretch the cables. We did not import crane clips.

The worker has no physics body. Walking is only a camera. Decorative ships do not collide with the playable ship: we wait a few seconds on SPACE instead.

7.1 Limitations and simplifications

Cables are stretched meshes. They do not go slack, and they are not ropes. The piston is the same idea: we scale a part between two anchors, we do not simulate hydraulics.

Water is a shader with a moving normal map. There is no fluid, and dropped cargo does not float. The dock is a large box plus fog. You cannot walk out to a real horizon. The worker stays inside a clamped rectangle. There is no score, no timer, and no second crane.

Shadow maps come only from the sun, and only in a fixed volume around the working area. A second shadow map at night was too expensive for the gain.

These cuts keep the browser usable. They also keep the work on the course topics: hierarchy, transforms, lights, textures, interaction, and animation written in JavaScript.

7.2 Conclusion

We wanted a harbor you can actually play with, not just look at. There is no score and no mission list. It is a sandbox: you move containers, call the ship, send the truck away, switch to night, walk around as a worker.

Even so, we tried to put in the main technical points from the course. The crane is a hierarchical model: we split it in Blender and animate it in JavaScript. The dock and roads use lights and textures. You interact with the crane, the slots and the physics when a box falls. The ship and the truck move on timed paths; the crane responds to held keys. We added day and night, fog, sound, and a few camera views so the port feels alive.